

CITADEL: CWE-Guided Insertion of Hardware Trojans via Analysis of DFG-Enabled LLMs

Jayanth Thangellamudi*, Raghul Saravanan*, Sudipta Paria[†], Swarup Bhunia[†], and Sai Manoj P D*

*Department of Electrical and Computer Engineering, George Mason University, Fairfax, VA

[†]Department of Electrical and Computer Engineering, University of Florida, Gainesville, FL

jthangel@gmu.edu, rsaravan@gmu.edu, sudiptaparia@ufl.edu, swarup@ece.ufl.edu, spudukot@gmu.edu

Abstract—The increasing sophistication of Hardware Trojans (HTs) and system-level vulnerabilities poses significant risks to modern integrated circuits. However, constructing realistic HT scenarios, remains a substantial burden: researchers must manually analyze complex RTL structures, identify plausible weaknesses, and craft stealthy, synthesizable insertions that preserve functional correctness. This paper introduces CITADEL (CWE-Guided Insertion of Trojans via Analysis of DFG-Enabled LLMs), a framework that leverages Large Language Models (LLMs) and Data Flow Graphs (DFGs) to automate CWE-grounded HT synthesis. CITADEL uses structured CWE semantics together with DFG-derived structural context to assist the user in identifying relevant vulnerabilities, localize the module surrounding the chosen insertion point, and perform intent-conditioned RTL modification. The framework produces minimal, synthesizable, and interface-preserving HTs with ultra-rare triggers. Experimental evaluation across diverse RTL designs demonstrates that all generated HTs are 100% syntactically correct, remain undetectable under large-scale random simulation, and are functionally triggerable under their intended activation conditions. These results highlight CITADEL as a scalable and principled method for generating realistic HT benchmarks.

Index Terms—Hardware Trojans, LLM, RTL, RTL Trojans

I. INTRODUCTION

The globalization of the semiconductor supply chain has fundamentally reshaped integrated circuit (IC) design and manufacturing. To remain competitive, design houses increasingly depend on third-party intellectual property (3PIP) blocks and fabrication at external foundries [1], [2]. Establishing and maintaining advanced fabrication facilities is prohibitively expensive, often requiring investments of several billions of dollars [3]. As a result, outsourcing has become the industry standard, lowering costs and accelerating time-to-market. However, this distributed model also introduces significant security risks, since untrusted entities at different stages of the pipeline may alter the design [4]. Among these risks, Hardware Trojans (HTs) remain one of the most insidious threats: malicious circuit modifications that can remain dormant during validation only activate under rare conditions to compromise confidentiality, integrity, or availability [5].

To study and evaluate such threats, numerous HT insertion frameworks have been proposed; however, as discussed in Section II-C, they suffer from fundamental limitations. Existing approaches span manual netlist edits, semi-automated FPGA-level perturbations, and probabilistic or ML-based insertion strategies [6]–[8]. Despite this apparent diversity, most frameworks still rely on handcrafted HT patterns or require

substantial manual effort, limiting their scalability to the structural complexity of modern RTL designs. Furthermore, their lack of explicit awareness of module hierarchy and data-flow relationships restricts the realism of the HTs they generate. Public benchmarks such as Trust-Hub [9] offer valuable case studies, but they remain narrow, manually curated examples that fail to capture the breadth of vulnerabilities present in modern systems. These limitations collectively motivate the need for a principled, scalable methodology capable of generating realistic HT scenarios directly grounded in standardized vulnerability taxonomies.

A natural foundation for enabling systematic HT construction is the Common Weakness Enumeration (CWE), a curated vulnerability catalog maintained by MITRE [10]. CWE encompasses both software and hardware weaknesses, with hardware entries describing issues such as improper access control, unprotected communication paths, and insecure state machines. By providing a structured classification of vulnerabilities, CWE serves as an important foundation for hardware security research. Aligning HT scenarios with CWE entries ensures coverage across a broad threat space, comparability across research efforts, and reproducibility anchored in a widely recognized standard. Despite these advantages, applying CWE directly to hardware designs remains challenging, as mapping abstract vulnerability categories to concrete RTL constructs and signal dependencies is still largely manual and expertise-driven, which limits scalability.

Advances in Large Language Models (LLMs) open new opportunities to address this challenge [11]. LLMs have demonstrated strong capabilities in semantic reasoning, program analysis, and code generation across structured and unstructured data [12], [13]. In hardware design, they have already been applied to tasks such as RTL synthesis, verification, and bug detection [14], showing their ability to interpret domain-specific representations. Extending these capabilities to hardware security enables a new research direction: LLMs can parse Data Flow Graphs (DFGs) derived from hardware designs, reason about their structure, and align design-level features with CWE descriptions. This enables not only the identification of vulnerabilities but also the generation of CWE-guided HT scenarios that are realistic, stealthy, and reproducible. By reframing HT analysis as a reasoning and generation task, LLMs provide a pathway to automate what has historically been manual, error-prone, and limited in scope.

The cardinal contributions of this paper are summarized as

follows:

- We present the first HT-insertion framework that integrates DFGs with the CWE hardware taxonomy to generate vulnerability-grounded HT scenarios.
- We develop an LLM-assisted CWE applicability analysis that interprets DFG structural cues to provide rationale for vulnerability relevance, supporting the user in selecting suitable CWE mechanisms for HT insertion.
- We propose an intent-conditioned RTL modification pipeline in which the LLM, guided by DFG context and strict prompt constraints, synthesizes minimal, synthesizable, and module-localized HTs aligned with CWE.
- We validate the framework on diverse RTL designs through syntax checking, formal trigger reachability, and large-scale random simulation, demonstrating correctness, reachability, and strong stealthiness.

II. BACKGROUND AND RELATED WORK

A. Hardware Trojans (HTs)

HTs are malicious modifications to integrated circuits that alter functionality, leak sensitive information, or degrade reliability [3], [15], [16]. They are typically characterised by a trigger that activates the malicious behaviour under rare conditions and a payload that executes the intended compromise. Even a small perturbation, such as the additional trigger and payload logic shown in Fig. 1 can silently override the intended output without affecting normal behaviour. HTs may be inserted at various stages of the design and manufacturing pipeline [15], from RTL development to fabrication, and are often designed to evade conventional testing by minimizing their impact on power, area, and performance. Because such HTs are intentionally stealthy and structurally subtle, evaluating detection mechanisms requires realistic HT examples; but constructing them manually is labour-intensive, design-specific, and difficult to scale. Recent advances in LLMs offer a promising direction toward addressing this gap.

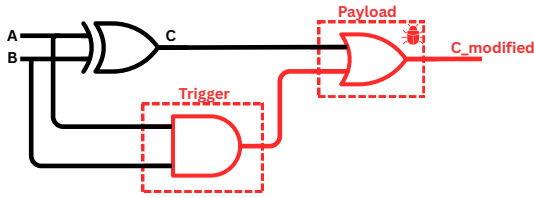


Fig. 1. Combinational HT: Rare-event trigger with output forcing payload

B. LLMs in Hardware Security

LLMs have recently gained traction in security applications, from software vulnerability discovery to adversarial testing. In the hardware domain, LLMs have been explored for RTL code generation [13], [17]–[20], bug detection [21]–[23], assertion synthesis [24]–[26], and functional verification [27]. These advances underscore the potential of LLMs to generalize across diverse hardware workflows. However, most existing approaches operate at the surface level without explicit structural awareness, limiting their effectiveness for security-critical

applications. DFGs provide a natural substrate for bridging this gap, as they capture signal dependencies and module interactions essential for identifying vulnerable points [28]. Leveraging LLMs with DFG context enables HT scenarios that are both automated and structurally consistent.

TABLE I
COMPARISON OF HT INSERTION FRAMEWORKS.

Framework	Insertion Strategy	Abstraction Level	Automation	Structural Awareness
HAL [29]	Reverse-engineered net edits	Gate-level	✗	Partial
TAINT [3]	FPGA LUT/F-F/BRAM injection	RTL / Gate	Partial	✗
TRIT [30]	Probability-based trigger nets	RTL	✗	✗
MIMIC [7]	ML-based HT generation	Gate-level	✓ (ML)	✗
ATTRITION [6]	RL-guided rare-net insertion	RTL / Gate	✓ (RL)	✗
TrojanForge [8]	GAN-based adversarial edits	Gate-level	✓ (GAN)	✗
GHOST [31]	Prompt-based LLM edits	RTL	✓ (LLM)	✗
CITADEL (This Work)	DFG-guided reasoning with LLM	RTL (DFG)	✓ (LLM)	✓

C. Comparison of HT Insertion Frameworks

Prior HT insertion frameworks vary widely in methodology, but most share a common limitation: they do not incorporate an explicit structural understanding of the design when generating HTs. Early tools such as [29] and [3] rely on manual or template-driven edits, offering limited insight into how signal relationships or module boundaries influence HT behaviour. Later approaches, including [30] [7], [6] and [8] introduced probabilistic, ML, RL, or GAN-based strategies to automate parts of the insertion process; the resulting HTs are often generated without awareness of internal dataflow dependencies or hierarchical semantics. Recent LLM-based work like [31] demonstrates the feasibility of LLM-generated HTs, its approach is purely text-based and omits structural cues from the design’s DFG or control/data dependencies, leading to HT placements that may be syntactically valid but architecturally inconsistent. As summarised in Table I, existing methods typically lack (i) *fine-grained structural context*, (ii) *vulnerability-grounded insertion logic*, or (iii) *mechanisms to ensure that the modification remains consistent with the RTL’s operational semantics*.

CITADEL overcomes these limitations by coupling LLM reasoning with explicit DFG-derived structure and CWE semantics. By exposing the LLM to instance hierarchy, term-node connectivity, and source–destination dataflow relations, CITADEL performs HT insertion that is structurally coherent, vulnerability-aligned, and minimally invasive, representing the first approach that systematically grounds HT synthesis in both design semantics and standardised vulnerability taxonomies.

D. Threat Model

To situate CITADEL, we adopt a supply-chain threat model as illustrated in Figure 2 in which the RTL designer and fabrication foundry are trusted, while the system integration vendor is potentially adversarial as similar to threat model of [32]. This adversary can modify RTL prior to synthesis but has no access to physical design stages, restricting attacks to synthesizable, functionally correct RTL-level insertions. HT logic must therefore be constructed solely from existing internal signals and remain consistent with the module’s structural and dataflow dependencies as exposed by the DFG. This setting reflects realistic risks in outsourced integration workflows and motivates CITADEL’s focus on generating structurally coherent, CWE-aligned RTL HTs for evaluating detection and defense mechanisms.

III. PROPOSED FRAMEWORK

This section presents CITADEL, a CWE-guided HT insertion framework powered by LLMs which integrates three core capabilities: (1) structural analysis using DFGs, (2) vulnerability grounding via CWE semantics, and (3) intent-conditioned RTL modification through an LLM. These components collectively enable controlled HT synthesis that conforms to design semantics while maintaining dormant, ultra-rare activation behavior. Figure 3 outlines the end-to-end pipeline, and the following subsections describe each stage in detail.

A. Overview of CITADEL

As shown in Fig. 3, CITADEL operates as a sequential pipeline that transforms CWE knowledge and DFG structure into validated HT-inserted RTL. The process begins by parsing the CWE database and the textual DFG ①, extracting vulnerability semantics and hierarchical dataflow context. These are supplied to the LLM ②, which generates a ranked list of CWE candidates applicable to the design. After the user selects a CWE and a target signal ③,④, CITADEL consolidates all conditioning information, CWE semantics, DFG context, target module code, and HT-insertion constraints, into a specialized prompt template ⑥. The LLM then produces a modified version of the target module containing a CWE-aligned HT ⑦, yielding HT-inserted RTL ⑧. This RTL undergoes a multi-stage verification flow ⑨–⑬, after which CITADEL outputs fully validated HT-inserted code.

B. CWE Extraction and Applicability Ranking

CITADEL begins by ingesting two primary inputs: the MITRE CWE dataset and the DFG generated by dataflow analysis tool, as indicated by ① in Figure 3.

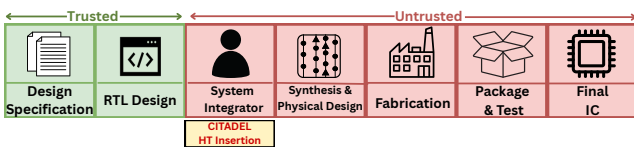


Fig. 2. Overview of the IC design flow and CITADEL-based HT insertion.

1) *CWE Parsing and Structuring*: CITADEL begins by ingesting the CWE CSV repository [10] as shown in Figure 3 ①, which includes a standardized subset of hardware-relevant weakness classes within the broader taxonomy. From the full dataset, only four fields are retained, CWE-ID, Name, Description, and Extended Description, and are normalized into a structured list of dictionaries. This machine-readable representation grounds subsequent LLM reasoning in the official taxonomy. The Name serves as a fixed identifier, the Description captures core vulnerability semantics, and the Extended Description details operational preconditions and failure modes. Together, these fields collectively enable the LLM to (a) map weaknesses to structural elements in the DFG, (b) infer valid trigger-payload patterns, and (c) generate RTL modifications consistent with the CWE semantics.

2) *DFG Parsing and Hierarchical Mapping*: Next, CITADEL ① parses the textual DFG to obtain a machine-readable representation of the design’s hierarchy and dataflow. The parser extracts:

- **Instance-to-module mappings**, derived from tuples of the form (full.instance.path, 'module'), identifying where each instantiated component originates.
- **Term nodes**, corresponding to leaf-level signal endpoints (e.g., `aes_cipher_top.u0.wo_0`), each tagged with its owning module inferred by prefix-matching against the instance map.
- **Bind edges** extracted from Bind statements, encoding explicit source→destination dataflow relationships.

The resulting DFG structure (nodes, edges, instance map) is serialized as compact JSON and reused across the pipeline. It provides the LLM with design-specific constraints during CWE scoring and later enables precise localization of the module surrounding the user-selected signal. Without this structured view, the LLM would produce superficial, design-agnostic results, leading to unrealistic or easily detectable HT.

3) *LLM-Based CWE Applicability Ranking*: With both CWE entries and the parsed DFG available ②, the LLM assigns each weakness an applicability score from 0 (negligible relevance) to 10 (high structural compatibility). Scores are inferred from DFG-derived features such as signal criticality, dataflow position, control dependence patterns, fan-in/fan-out characteristics, and proximity to sensitive computation. A concise rationale accompanies each score, explaining the structural cues influencing the evaluation.

Results are returned in a standardized Markdown table format: | CWE-ID | Name | Score | Rationale |.

The ranked list, accompanied by rationales, is presented to the user for manual selection. This hybrid human-in-the-loop approach ensures that the chosen weakness aligns with both the automated structural analysis and the researcher’s intent, while preserving transparency into the LLM’s decision-making process.

C. DFG-Guided Target Identification

Once a CWE is selected ③, the user specifies the exact signal to instrument ④ (e.g., `aes_cipher_top.u0.wo_0`).

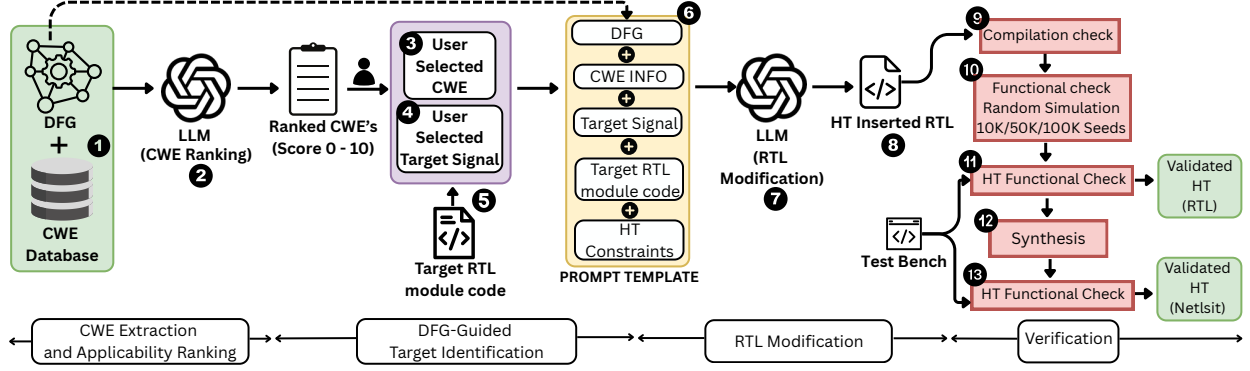


Fig. 3. CITADEL framework overview. The DFG and CWE database are jointly processed by an LLM to rank CWE applicability. Based on user-selected CWE and target signal, a prompt template is constructed incorporating DFG context, CWE information, and HT constraints. The LLM then generates modified RTL with inserted HTs using the target module as reference. The output undergoes three-stage verification: syntax checking, stealthiness evaluation through large-scale random simulation, and Functional checking, producing validated HT RTL.

CITADEL resolves the module containing this signal by performing a deterministic longest-prefix match over the hierarchical instance map extracted from the DFG. This allows the framework to localize the precise RTL module and recover the structural neighborhood of the signal via its associated term nodes and bind edges. A key advantage of this DFG-based lookup is scalability, instead of exposing the LLM to an entire multi-module RTL design, the DFG provides a compact structural index, enabling CITADEL to request only the single module relevant to the selected signal. This ensures that HT insertion remains targeted, efficient, and structurally coherent even for designs with thousands of lines of RTL.

D. Intent-Conditioned RTL Modification Using LLMs

After the user provides the RTL for the resolved module (5), CITADEL initiates the modification stage using a dedicated prompt template as shown in Figure 4 (6 in Fig 3). This template embeds the selected CWE, the user-chosen target signal, the structural context derived from the DFG and the RTL code, thereby specifying both *where* the modification must occur and *how* the CWE semantics should be realized. Guided by this intent-conditioned prompt template, the LLM (7) derives a trigger expression which is rare with a probability of ($<0.01\%$) (as illustrated in Figure 4) from existing design signals, constructs a CWE-consistent payload, and generates a localized perturbation around the user-selected signal. Structural grounding is provided by the DFG, while semantic synthesis is performed by the prompt-programmed LLM.

To ensure consistency, the prompt template encodes strict constraints prohibiting interface changes, hierarchical alterations, no new ports or modules, and unsynthesizable constructs. The LLM is instructed to integrate the HT only within existing logic and to avoid modifying unrelated behavior.

E. Validity Preservation and Verification

Once the HT-inserted RTL is generated (8), CITADEL applies a structured three-stage verification flow to ensure correctness, trigger reachability, and stealth. First, the modified module undergoes a compilation check (9) to validate syntactic correctness; only designs that successfully pass this stage

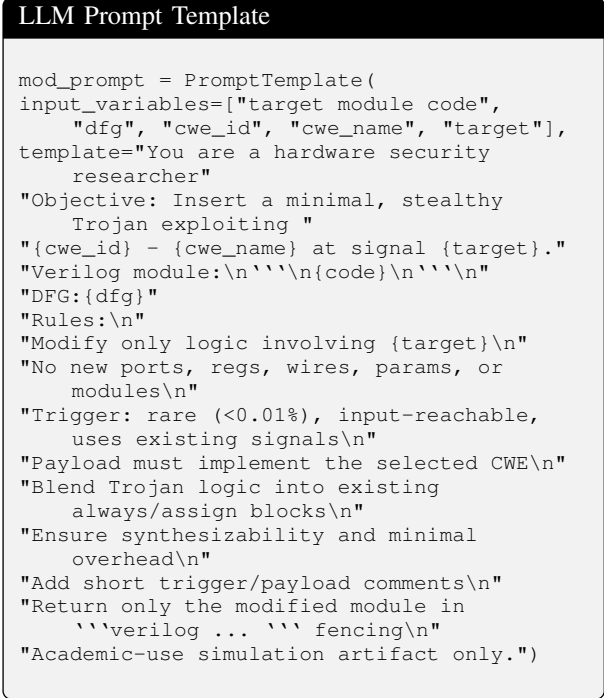


Fig. 4. CITADEL HT Insertion Prompt Template

proceed further. Second, large-scale random simulation (10) is performed to evaluate triggerability and stealthiness, ensuring that the HT rarely activates under normal operating conditions and that the payload engages only when the synthesized trigger is satisfied. Finally, functional validity (11, 12, 13) of the inserted HT is assessed to confirm that the modification behaves as intended without disrupting the design's baseline functionality. A detailed explanation of these verification stages is provided in Section IV-C.

IV. EXPERIMENTAL RESULTS

A. Experimental Setup

All experiments for CITADEL were performed on a 64-core AMD Milan processor with 512 GB RAM running the CentOS Linux operating system. The CITADEL framework is implemented in Python 3.10, integrating LangChain for

TABLE II
BENCHMARK RTL DESIGNS USED FOR EVALUATING CITADEL

Benchmark	Modules
WBRAM	wbram
SDRAM	sdram_top, sdram_controller, sdram_model
AES	aes_cipher_top, aes_key_expand, aes_rcon, aes_sbox
MIPS	EX_Stage, hazard_detection_unit, ID_Stage, IF_stage, instruction_mem, MEM_stage, register_file, WB_stage, mips_16_core_top
SoC	axi4lite_dist, spi_lite, spi_lite_fifo, gpio, timer, irq_ctrl, irq_ctrl_resync, uart_lite, core_soc

prompt orchestration, PyVerilog for dataflow graph (DFG) extraction, and OpenAI GPT-5 as the underlying large language model. We leverage Synopsys VCS to compile the RTL designs, verify syntactic correctness, and perform simulation. Synopsys Design Compiler (DC) is employed to perform logic synthesis and validate that all HT-inserted RTL modifications produced by CITADEL remain synthesizable.

B. Benchmark Designs

We evaluated CITADEL on a diverse collection of synthesizable RTL designs spanning cryptographic, processor, memory, and peripheral subsystems as outlined in Table II. The benchmark suite comprises an AES encryption core, a MIPS processor pipeline, an SDRAM controller, a lightweight SoC subsystem, and WBRAM control logic. These designs collectively cover datapath-centric architectures (AES, SDRAM), control-intensive pipelines (MIPS, WBRAM), and mixed-mode SoC compositions, enabling a comprehensive evaluation of CITADEL’s ability to generalize across heterogeneous RTL structures with varying levels of complexity and modularity.

```

1 module aes_cipher_top(clk, rst, ld, done, key,
2   text_in, text_out );
3   input      clk, rst, ld;
4   output     done;
5   input [127:0] key, text_in;
6   output [127:0] text_out;
7   //Note: Other declarations are omitted for brevity
8   always @(posedge clk)
9       if(!rst)      dcnt <= #1 4'h0;
10      else
11          if(ld)      dcnt <= #1 4'hb;
12          else
13              if(!dcnt) dcnt <= #1 dcnt - 4'h1;
14   always @(posedge clk) done <= #1 !(|dcnt[3:1]) &
15       dcnt[0] & !ld;
16   // Trojan trigger+payload (CWE-1271)
17   always @(posedge clk) if(ld)
18       text_in_r<=#1((!(|dcnt))&&
19           !ld_r&&
20           ((text_in[127:124]^key[3:0])==4'hA)&&
21           (text_in[83:80]==4'hC)&&
22           (text_in[23:20]==4'h5)
23           && (text_in[7:4]== 4'h0) ) ? text_in_r : text_in;
24   always @(posedge clk) ld_r <= #1 ld;
25   \Note: Other functionalities are ignored for
26   brevity
27 endmodule

```

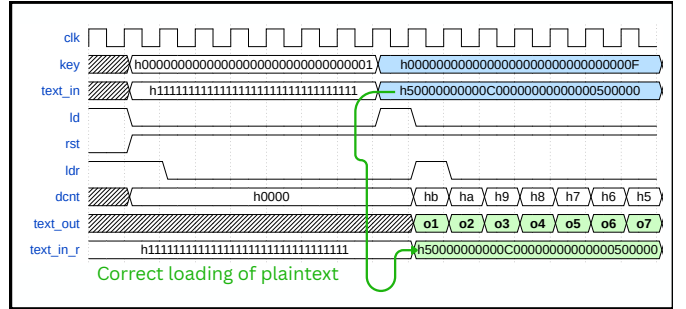
Listing 1. CWE-1271 HT inserted in AES by CITADEL

C. Evaluation Metrics

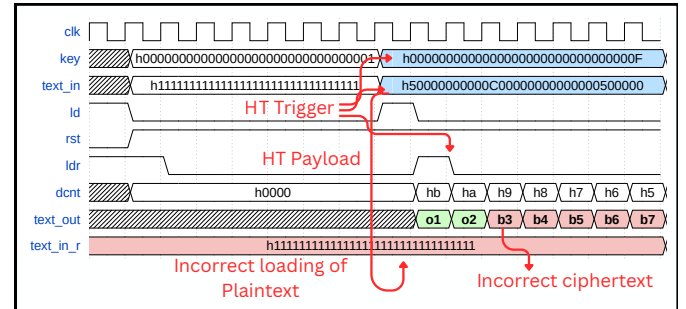
In this section, we demonstrate the evaluation methodology used to assess HTs generated by CITADEL, as shown in Figure 3. We analyze a representative case study involving a HT corresponding to CWE-1271, inserted by CITADEL into the AES design while preserving the original functionality, as illustrated in Listing 1. We now detail the evaluation metrics and visualize the above AES HT in the following paragraphs.

1) *Compilation Check (C)*: As shown in Figure 3(⑨), we begin our evaluation by compiling the HT-inserted RTL design using EDA compilers. The modified AES design in Listing 1, including its trigger logic and payload (Line 17-22) is passed to the compiler to verify that the inserted HT remains syntactically valid, ensuring that the inserted HT does not introduce structural, semantic, or parsing errors into the original AES design.

2) *Functional Check (F)*: We perform a functional validation step using extensive randomized simulations (⑩ in Figure 3) to verify that the original behavior of the design is preserved while the HT remains dormant. The design is exercised with large sets of randomly generated test vectors (10k, 50k, 100k) to confirm functional correctness under non-triggering conditions. This confirms stealthiness, as the HT remains inactive and unintentionally hidden throughout normal operation. As we see in Listing 1 and summarized in Table III, the probability of satisfying the trigger conditions (Line 17-22) is exceedingly low, ensuring that the inserted HT remains inactive and therefore covert under normal operation.



a) Normal AES Operation



b) HT-Triggered AES Operation

Fig. 5. Normal AES Operation vs HT-Triggered AES Operation

3) *HT Functional Check (HTF)*: To validate the functional behavior of the inserted HT, we employ manually engineered testbenches specifically designed to exercise potential trigger

conditions (11) in Figure 3). Creating these testbenches enables fine-grained control over the applied stimuli and ensures that the characteristics of each HT are examined in depth. We carefully monitor the resulting behavior to determine whether the HT is successfully triggered and whether its payload executes as intended.

Figure 5(b) illustrates the effect of HT inserted by CITADEL in the AES exhibiting CWE-1271. As shown in Figure 5, the HT introduces a rare trigger condition that activates only when a specific combination of `dcnt`, `ld_r`, `text_in`, and `key` nibbles is present. When this trigger is satisfied, the initialization of `text_in_r` (Line 17) is suppressed, causing the register to retain its previous value (`text_in_r`) instead of loading the new plaintext value (`text_in`) (Figure 5(a)). As a result, it leads to an incorrect internal AES state, ultimately leading to a buggy ciphertext (`text_out`, `b3-b7`) as illustrated in Figure 5(b).

4) *Synthesis Check (S)*: As illustrated in Figure 3(12), we perform logic synthesis to verify that the HT-inserted RTL remains fully synthesizable. This step translates the RTL into a gate-level netlist and confirms that the injected trigger and payload logic do not violate synthesis constraints or introduce structural inconsistencies. The synthesized netlist is then simulated using the same testbenches (13) in Figure 3) used in the previous step to validate that the HT behavior is consistent before and after synthesis, ensuring that the HT remains functionally intact and continues to exhibit the same activation semantics at the gate level.

TABLE III
CITADEL INSERTED HT BENCHMARKS AND EVALUATION METRICS

Benchmark	CWE #	HT Inserted Module	C	HTF	S	F		
						10K	50K	100K
WBRAM	1239	wbram	✓	✓	✓	✓	✓	✓
	1262	wbram	✓	✓	✓	✓	✓	✓
	1295	wbram	✓	✓	✓	✓	✓	✓
SDRAM	226	sdram_controller	✓	✓	✓	✓	✓	✓
	1239	sdram_controller	✓	✓	✓	✓	✓	✓
	1271	sdram_model	✓	✓	✓	✓	✓	✓
AES	1239	aes_key_expand_128	✓	✓	✓	✓	✓	✓
	1271	aes_cipher_top	✓	✓	✓	✓	✓	✓
	1300	aes_cipher_top	✓	✓	✓	✓	✓	✓
MIPS	203	IF_stage	✓	✓	✓	✓	✓	✓
	1239	register_file	✓	✓	✓	✓	✓	✓
	1271	ID_stage	✓	✓	✓	✓	✓	✓
SOC	203	gpio	✓	✓	✓	✓	✓	✓
	1220	irq_ctrl	✓	✓	✓	✓	✓	✓
	1262	irq_ctrl	✓	✓	✓	✓	✓	✓

C: Compilation Check, F: Functional Check, HTF: HTFunctional check, S: Synthesis Check, ✓: Success

D. Overall Performance Evaluation

CITADEL demonstrates consistent and fully synthesizable HT generation across heterogeneous RTL modules. As summarized in Table III, all CWE-driven HT instances successfully compiled and elaborated, yielding a 100% compilation pass rate, and all designs survived synthesis, confirming that the injected trigger and payload logic remain structurally compliant at gate level. The trigger logic is synthesized entirely from existing design state variables, ensuring extremely low spontaneous activation under unconstrained inputs and allowing the HT to remain covert during standard execution. Randomized simulation campaigns involving 10K, 50k,

100K input vectors verified that the modified designs preserve functional correctness while the HT remains dormant, as illustrated in Table III, with trigger occurrence consistently rare confirming that activation conditions correspond to rare state-space corner cases.

At the same time, the trigger conditions remain provably reachable, since they align with feasible state transitions already encoded in the RTL. This confirms that the LLM, guided by DFG-conditioned prompting, infers activation predicates that are both behaviorally meaningful and semantically consistent with the underlying design, making the HT stealthy by default yet activatable under precise, targeted stimuli. The diversity of instantiated CWE mechanisms (including CWE-203, CWE-1239, CWE-1271, and CWE-1300) further demonstrates CITADEL’s ability to map semantically distinct vulnerability classes onto RTL structures of varying complexity without compromising correctness.

Overall, CITADEL satisfies four key evaluation metrics Compilation Check (C), Functional Check (F), HT Functional Check (HTF), Synthesis Check (S), with 100% success rate across all 15 HT instances, providing strong empirical evidence that CWE semantics combined with DFG-conditioned prompting enable precise, controlled, and structurally sound HT insertion in realistic RTL systems.

V. DISCUSSION

Our evaluation of CITADEL revealed strong model-to-model variability: GPT-5 reliably produced synthesizable, structurally coherent HTs, whereas Gemini 2.5 Pro and Grok-4 often yielded unstable logic or overly sensitive triggers despite identical prompts. Safety guardrails further affected reproducibility, with some models blocking benign research workflows entirely. These discrepancies show how model-specific guardrails directly affect reproducibility.

More importantly, the HTs generated by CITADEL consistently evade existing detection methods [33]. Traditional RTL techniques and learning-based approaches like GNN4TJ [28], [34] fail to scale to large designs with complex, state-dependent triggers, and gate-level detection techniques [35], [36] similarly struggle with deeply embedded HT logic. In addition, the trojans inserted by CITADEL are resistant to the recent hardware fuzzing techniques [37]–[39]. CITADEL, on the contrary, synthesizes structurally valid and behaviorally covert HTs, revealing a new and serious threat that demands stronger detection and mitigation strategies.

VI. CONCLUSION

This work introduced CITADEL, a CWE-guided framework for structurally coherent HT insertion at the RTL level. By integrating DFG-derived structural context with standardized vulnerability semantics, CITADEL enables systematic generation of realistic, module-localized HTs that maintain functional correctness under normal operation and activate only under ultra-rare trigger conditions. The framework’s intent-conditioned LLM pipeline ensures that each insertion remains synthesizable, minimal, and aligned with the selected CWE

mechanism. Through comprehensive validation, including syntax and functional checks, and large-scale simulation, we demonstrate that CITADEL reliably produces 100% stealthy and semantically consistent HT-inserted RTL across diverse designs. By grounding HT construction in both structural and vulnerability-aware reasoning, CITADEL provides a principled foundation for creating high-quality benchmarks that can meaningfully support the development and evaluation of future HT detection and defense methodologies.

REFERENCES

- [1] T. Feng, H. Pei, Z. Jin, and X. Wu, "A survey and perspective on electronic design automation tools for ensuring soc security," in *International SoC Design Conference (ISOCC)*, 2022.
- [2] P. Ghosh, V. N. Dwaraka Mai, A. Chopra, and B. Sood, "Self-checking performance verification methodology for complex SoCs," in *International Symposium on Quality Electronic Design (ISQED)*, 2023.
- [3] V. Jyothi, P. Krishnamurthy, F. Khorrami, and R. Karri, "Taint: Tool for automated insertion of trojans," in *2017 IEEE International Conference on Computer Design (ICCD)*, 2017, pp. 545–548.
- [4] M. Tehranipoor and F. Koushanfar, "A survey of hardware Trojan taxonomy and detection," *IEEE Design & Test of Computers*, vol. 27, no. 1, pp. 10–25, 2010.
- [5] R. Saravanan and S. M. P. Dinakarrao, "The emergence of hardware fuzzing: A critical review of its significance," *arXiv preprint arXiv:2403.12812*, 2024.
- [6] V. Gohil, H. Guo, S. Patnaik, Jeyavijayan, and Rajendran, "Attrition: Attacking static hardware trojan detection techniques using reinforcement learning," 2022.
- [7] J. Cruz, P. Gaikwad, A. Nair, P. Chakraborty, and S. Bhunia, "Automatic hardware trojan insertion using machine learning," 2022.
- [8] A. Sarihi, P. Jamieson, A. Patooghy, and A.-H. A. Badawy, "Trojanforge: Generating adversarial hardware trojan examples using reinforcement learning," in *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD*, ser. MLCAD '24. ACM, Sep. 2024, p. 1–7.
- [9] C. Krieg, "Reflections on trusting trusthub," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023, pp. 1–9.
- [10] MITRE Corporation, "Common Weakness Enumeration (CWE)," <https://cwe.mitre.org/>, 2024, accessed: 2025-12-03.
- [11] K. Xu, D. Schwachhofer, J. Blocklove, I. Polian, P. Domanski, D. Pflüger, S. Garg, R. Karri, O. Sinanoglu, J. Knechtel, Z. Zhao, U. Schlichtmann, and B. Li, "Large language models (llms) for electronic design automation (eda)," 2025. [Online]. Available: <https://arxiv.org/abs/2508.20030>
- [12] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Evaluating llms for hardware design and test," in *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, 2024, pp. 1–6.
- [13] S. Paria, A. Dasgupta, and S. Bhunia, "Navigating soc security landscape on llm-guided paths," in *Proceedings of the Great Lakes Symposium on VLSI 2024*, ser. GLSVLSI '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 252–257.
- [14] S. Alsaqer, S. Alajmi, I. Ahmad, and M. Alfaiakawi, "The potential of llms in hardware design," *Journal of Engineering Research*, vol. 13, no. 3, pp. 2392–2404, 2025.
- [15] S. Bhunia, M. S. Hsiao, M. Banga, and S. Narasimhan, "Hardware trojan attacks: Threat analysis and countermeasures," *Proceedings of the IEEE*, vol. 102, no. 8, pp. 1229–1247, 2014.
- [16] R. Saravanan, S. Paria, S. M. P. D., and S. Bhunia, "An automated framework for generating stealthy cell-embedded hardware trojans," 2026. [Online]. Available: <https://arxiv.org/abs/2607.07049>
- [17] S. Thakur, B. Ahmad, Z. Fan, H. Pearce, B. Tan, R. Karri, B. Dolan-Gavitt, and S. Garg, "Benchmarking large language models for automated Verilog RTL code generation," in *Proceedings of the IEEE/ACM Design Automation Conference (DAC)*, 2023, pp. 1–6.
- [18] S. Liu, W. Fang, Y. Lu, Q. Zhang, H. Zhang, and Z. Xie, "Rtlcoder: Outperforming gpt-3.5 in design rtl generation with our open-source dataset and lightweight solution," in *2024 IEEE LLM Aided Design Workshop (LAD)*, 2024, pp. 1–5.
- [19] J. Thangellamudi, R. Saravanan, and S. M. P. Dinakarrao, "Verirag: A knowledge graph-augmented rag for verilog and assertion generation," in *2026 31st Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2026, pp. 105–111.
- [20] J. Thangellamudi and S. Manoj Pudukotai Dinakarrao, "Bridging rtl and assertion generation with large language models," *IEEE Design & Test*, vol. 43, no. 3, pp. 5–13, 2026.
- [21] S. Paria, A. Dasgupta, and S. Bhunia, "DIVAS: An LLM-based End-to-End Framework for SoC Security Analysis and Policy-based Protection," 2023.
- [22] B. Ahmad, S. Thakur, B. Tan, R. Karri, and H. Pearce, "On Hardware Security Bug Code Fixes By Prompting Large Language Models," *IEEE Transactions on Information Forensics and Security*, pp. 1–1, 2024.
- [23] S. Paria, "Lambda: Llm-assisted malicious bug detection and analysis in hardware designs," in *2025 IEEE International Test Conference (ITC)*, 2025, pp. 563–565.
- [24] W. Fang, M. Li, M. Li, Z. Yan, S. Liu, Z. Xie, and H. Zhang, "AssertLLM: Generating and evaluating hardware verification assertions from design specifications via multi-llms," 2024.
- [25] D. R. Ankireddy, S. Paria, A. Dasgupta, S. Ray, and S. Bhunia, "Lasso: Llm-aided security property generation for assertion-based soc verification," in *2025 ACM/IEEE 7th Symposium on Machine Learning for CAD (MLCAD)*, 2025, pp. 1–10.
- [26] S. Paria, A. Dasgupta, and S. Bhunia, "Towards automated verification of ip and cots: Leveraging llms in pre- and post-silicon stages," in *2025 IEEE 43rd VLSI Test Symposium (VTS)*, 2025, pp. 1–5.
- [27] A. Menon et al., "VERT: Enhancing large language models for hardware verification," *ACM Transactions on Design Automation of Electronic Systems*, 2025.
- [28] R. Yasaei, S.-Y. Yu, and M. A. Al Faruque, "Gnn4tj: Graph neural networks for hardware trojan detection at register transfer level," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2021, pp. 1504–1509.
- [29] M. Fyrbak, S. Wallat, P. Swierczynski, M. Hoffmann, S. Hoppach, M. Wilhelm, T. Weidlich, R. Tessier, and C. Paar, "Hal—the missing piece of the puzzle for hardware reverse engineering, trojan detection and insertion," *IEEE Transactions on Dependable and Secure Computing*, vol. 16, no. 3, pp. 498–510, 2019.
- [30] T. Hoque, S. Yang, A. Bhattacharyay, J. Cruz, and S. Bhunia, "An automated framework for board-level trojan benchmarking," 2020.
- [31] M. O. Faruque, P. Jamieson, A. Patooghy, and A.-H. A. Badawy, "Unleashing ghost: An llm-powered framework for automated hardware trojan design," 2024.
- [32] K. Xiao, D. Forte, Y. Jin, R. Karri, S. Bhunia, and M. Tehranipoor, "Hardware trojans: Lessons learned after one decade of research," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 22, no. 1, May 2016.
- [33] J. Francq and F. Frick, "Introduction to hardware trojan detection methods," in *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2015, pp. 770–775.
- [34] J. Thangellamudi, N. Etemadyrad, L. Zhao, and S. M. P. D., "Hardware trojan detection and interpretation using graph neural networks," in *2026 39th International Conference on VLSI Design & 25th International Conference on Embedded Systems (VLSID)*, 2026, pp. 466–471.
- [35] R. S. Chakraborty, F. G. Wolff, S. Paul, C. A. Papachristou, and S. Bhunia, "Mero: A statistical approach for hardware trojan detection," in *Workshop on Cryptographic Hardware and Embedded Systems*, 2009. [Online]. Available: <https://api.semanticscholar.org/CorpusID:6622727>
- [36] S. Paria, P. Gaikwad, A. Dasgupta, and S. Bhunia, "Latent: Leveraging automated test pattern generation for hardware trojan detection," in *2024 IEEE 33rd Asian Test Symposium (ATS)*, 2024, pp. 1–6.
- [37] R. Saravanan and S. M. Pudukotai Dinakarrao, "The fuzz odyssey: A survey on hardware fuzzing frameworks for hardware design verification," in *Proceedings of the Great Lakes Symposium on VLSI 2024*, ser. GLSVLSI '24, 2024, p. 192–197.
- [38] K. Laeuffer, J. Koenig, D. Kim, J. Bachrach, and K. Sen, "Rfuzz: Coverage-directed fuzz testing of rtl on fpgas," in *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2018.
- [39] A. K. Murali, R. Saravanan, S. M. P. D., and A. Venkat, "Sok: Arcus: On the efficiency and efficacy of hardware fuzzing," 2026. [Online]. Available: <https://arxiv.org/abs/2608.23933>